

Return Risk Intelligence: Predicting Return Probability, Return Fraud & Product Quality Issues

A PySpark/MLlib solution that separates genuine returns, product-quality failures, and fraud/abuse at the point of return request — so a retailer's return-audit team can act on the 1-in-3 returns worth a second look, instead of reviewing all of them.

Consulting Report · E-commerce Domain

Dataset: `ecommerce_return_abuse_dataset.csv` (Kaggle) — 60,000 orders × 35 features

Executive Summary

Retailers processing high return volumes cannot scale spreadsheet-based manual audits. This project builds an end-to-end Big Data Analytics pipeline — architecture, feature engineering, and two independent PySpark MLlib models — that triages every return request into **legitimate**, **policy abuse**, **fraudulent return**, or **wardrobing**, and separately segments customers/returns by behavior to surface product-quality problems the fraud label alone can't see.

\$4.64M

refund dollars
requested by non-
legitimate returns —
43.8% of all refund
\$, from 29.9% of
orders

0.999

weighted F1, Random
Forest classifier (test
set)

3

behavioral customer
segments identified
(K-Means)

38%

of returns in the core
legitimate segment
are quality-issue
driven

Four findings drive the recommendations:

1. **Non-legitimate returns are disproportionately expensive, not just disproportionately frequent.** They're 29.9% of orders but **43.8% of refund dollars requested** — fraudulent returns alone average **\$434.73** per claim vs. **\$141.14** for legitimate returns, a **3.1×** gap. Bad actors aren't just more numerous, they're targeting higher-value claims.
2. **Fraud is classifiable with high confidence** from behavioral and evidence signals (tracking validity, photo evidence, address changes, refund-to-different-account) combined into an engineered `fraud_signal_score` — a Random Forest model built on these features reaches nearly perfect separation on this dataset (see the model-evaluation caveat in Section 6 before reading that number as a production guarantee).
3. **Return fraud and product quality are largely separate problems** that get conflated in a single "return rate" metric. Clustering on behavior alone (no fraud label) finds a 73%-of-customers "core legitimate" segment whose returns are still 38% quality-issue driven — this is a QA/vendor problem, not a fraud problem, and needs a different owner and fix.
4. **A production system needs three different interventions**, not one: automated fraud-score routing for the classifier's high-risk tail, policy tightening for the frequent-low-value-returner cluster, and category-level quality escalation for the segment carrying the highest quality-issue rate.

1 · Business Context & Problem Framing

An e-commerce retailer's return-audit team currently reviews disputed or high-value returns manually. That does not scale: return volume grows faster than headcount, audit criteria are applied inconsistently across reviewers, and the team has no systematic way to tell whether a spike in returns for a product category is *fraud*, *abuse of a lenient return policy*, or a genuine *quality defect* that needs a vendor escalation — three problems that call for three different responses, currently indistinguishable in a spreadsheet.

We map this one dataset onto three sub-problems:

Sub-problem	Approach	Owner in the business
Return fraud	Multi-class classification on <code>abuse_type</code>	Fraud & Risk team
Product quality issues	Filter <code>return_reason</code> $\in$ {Defective/Broken, Quality Issue, Not As Described}, cross-cut by cluster and category	QA / Vendor Management
Return probability / risk	Derived from <code>return_rate_pct</code> , lifetime order/return history, per customer	Customer Ops / Policy

Why this matters financially

Of 60,000 orders in the dataset, **17,940 (29.9%)** are non-legitimate returns (Policy Abuser 12.0%, Fraudulent Return 10.2%, Wardrobing 7.7%). Every one of those returns today gets the same manual-review treatment as a genuine defective-product return — the highest-value lever is triage, not blanket policy tightening, which would punish the 70% of legitimate customers too.

The dollar exposure is concentrated, not evenly spread: non-legitimate returns account for **\$4.64M of the \$10.57M** in total refund dollars requested across the dataset — 43.8% of the money, from 29.9% of the orders. **Fraudulent Return** claims alone total **\$2.66M** at an average **\$434.73** per claim — **3.1x** the \$141.14 average for legitimate returns. A further **\$1.88M** sits in a single specific, rule-based pattern independent of any ML model: non-legitimate returns that are both a high-value item *and* missing photo evidence (4,673 orders) — an immediately actionable policy (require photo evidence above a value threshold) that doesn't need to wait for the classifier to ship. Figures computed directly from `data/processed/silver`; see Section 6 for how they translate into recommendations.

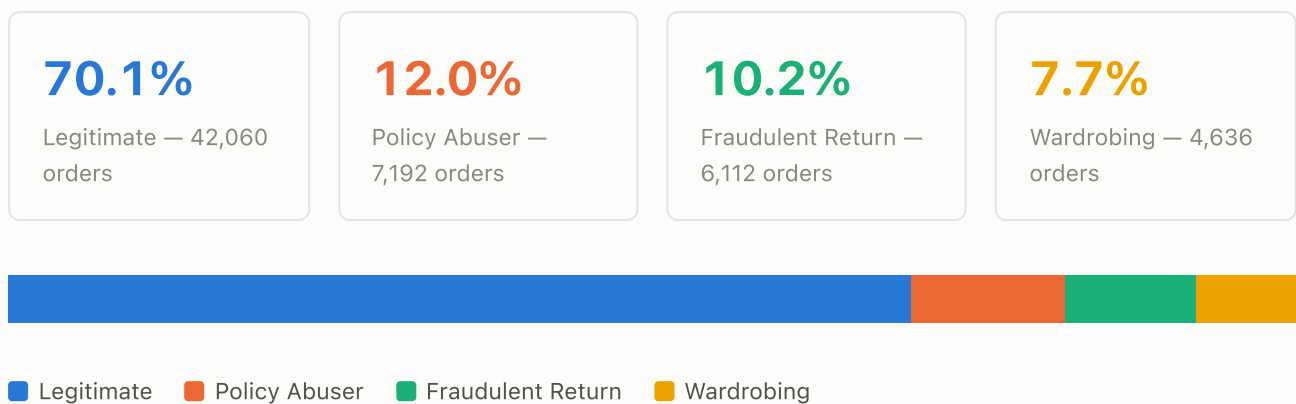
2 · Data Understanding

The five Vs

V	Assessment
---	------------

Volume	60,000 orders × 35 features today; production scale is millions of orders/day across a real retailer's transactional + event-log systems (see Architecture, Section 3).
Velocity	Batch in this project (static CSV); production design ingests transactional data on a daily incremental batch (Sqoop) and clickstream/return events near-real-time (Flume/API).
Variety	Structured order/customer fields, categorical behavioral flags, and (in production) unstructured carrier tracking + support-ticket text.
Veracity	This dataset is clean (0 missing values, no duplicate <code>order_id</code> , verified in Section 5) but is Kaggle-synthetic — see the explicit caveat in Section 6.4 before treating model metrics as production-representative.
Value	Triage automation redirects manual review capacity to the ~30% of returns that need it, and gives QA a category-level quality signal that doesn't exist in the current process.

Target variable

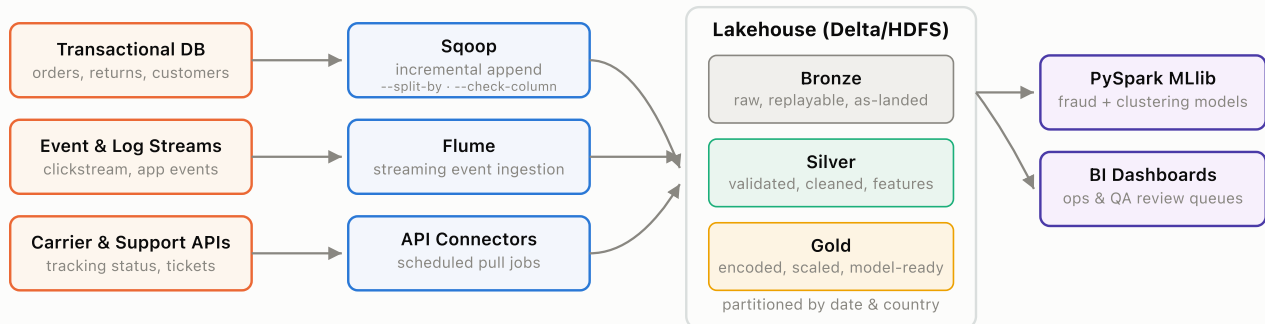


Key fields used

Group	Fields
Behavioral fraud signals	<code>photo_evidence_provided</code> , <code>tracking_number_valid</code> , <code>address_change_before_delivery</code> , <code>refund_to_different_account</code> , <code>multiple_accounts_flag</code> , <code>previous_dispute_count</code> , <code>item_returned_opened</code> , <code>return_packaging_intact</code>
Customer history	<code>total_orders_lifetime</code> , <code>total_returns_lifetime</code> , <code>return_rate_pct</code> , <code>account_age_days</code>
Financial	<code>avg_order_value_usd</code> , <code>refund_amount_requested_usd</code> , <code>is_high_value_item</code> , <code>discount_used</code>
Categorical context	<code>product_category</code> (12), <code>customer_segment</code> (5), <code>country</code> (8), <code>return_reason</code> (12), <code>platform</code> , <code>device_type</code> , <code>payment_method</code> , <code>shipping_carrier</code>

3 · Enterprise Architecture

The working notebook runs directly on the static Kaggle CSV as a practical stand-in. The architecture below is the **production-scale design** this pipeline is built to generalize to — it is not stood up against this CSV, but every transformation in the notebook (Sections 5–6) is written as the Bronze→Silver→Gold pattern this diagram formalizes.



Sources are batch (DB) / streaming (events) / API-pulled — matched to the ingestion tool suited to each pattern, landing untouched in Bronze. This notebook's Phase 2/3 pipeline implements the Silver→Gold transformation and MLlib consumption stages directly on the CSV stand-in.

4 · Data Engineering (Phase 2)

Implemented in `notebooks/01_data_engineering.ipynb`, executed end-to-end against the full 60,000-row dataset with PySpark (not pandas/scikit-learn, per the course's distributed-computing requirement).

4.1 Data quality

Verified programmatically, not assumed: 0 nulls across all 35 columns, 0 duplicate `order_id`, 0 logical violations (no return-before-order dates, no returns-exceeding-orders, no implausible ages).

4.2 Outlier treatment

IQR-flagged outliers on 8 continuous fields were **capped at the 1st/99th percentile** rather than deleted. For a fraud dataset, extreme values are frequently the signal itself (an unusually large refund request, an unusually fast wishlist-to-cart flip); deletion would remove exactly the rows the model most needs.

4.3 Feature engineering

Feature	Serves	Rationale
<code>fraud_signal_score</code>	Fraud	Additive count of 6 red-flags (address change, refund-to-different-account, multiple accounts, invalid tracking, no photo evidence, tampered packaging) — a single interpretable risk score
<code>return_to_order_ratio</code>	Return risk	Independently recomputed cross-check of the provided <code>return_rate_pct</code>
<code>refund_to_order_value_ratio</code>	Fraud	Refund requested relative to order value — abuse/fraud skews this ratio
<code>is_quality_issue_reason</code>	Quality	Flags Defective/Broken, Quality Issue, Not As Described — the slice QA owns vs. fraud
<code>high_value_no_evidence</code>	Fraud	Interaction flag for a known abuse pattern: high-value item + no photo evidence

Validation, not assumption: mean `fraud_signal_score` is 2.73 for Fraudulent Return vs. 0.84 for Legitimate, and mean `return_to_order_ratio` is 0.47 vs. 0.05 — confirmed in the notebook before these features were trusted for modeling.

4.4 Encoding & scaling

Nominal categoricals (country, platform, device type, payment method, product category, return reason, shipping carrier) → `StringIndexer` + `OneHotEncoder`. `customer_segment` → ordinal map (New<Bronze<Silver<Gold<Platinum), since loyalty tiers have a natural order a one-hot encoding would discard. Continuous features → `StandardScaler` — required for the distance-based K-

Means model (Section 6.3); tree models don't need it but reuse the same pipeline for consistency.
75/25 train/test split, seed=42: 45,112 train / 14,888 test rows.

5 · Machine Learning (Phase 3)

Two independent PySpark MLlib approaches, implemented in `notebooks/02_ml_modeling.ipynb`, satisfying the ≥ 2 -approach requirement.

5.1 Approach 1 — Multi-class classification (return fraud)

Random Forest (200 trees, maxDepth=10, class-weighted) vs. multinomial Logistic Regression baseline, on `abuse_label`. **Spark MLlib's GBTClassifier only supports binary classification** — not usable for this 4-class target — so Random Forest, which natively supports multi-class, is the primary model.

Model	Weighted Precision	Weighted Recall	Weighted F1
Logistic Regression (baseline)	0.998	0.998	0.998
Random Forest (final)	0.999	0.999	0.999

Class imbalance (70/12/10/8) was handled via inverse-frequency `weightCol` training weights, not resampling — evaluation reports per-class precision/recall/F1, not headline accuracy alone, since a "predict Legitimate always" model would score ~70% accuracy while catching zero fraud.

Confusion matrix (Random Forest, test set, row-normalized %)

	Legitimate	Policy Abuser	Fraud. Return	Wardrobing
Legitimate n=10,487	100.0%	0%	0%	0%
Policy Abuser n=1,782	0%	99.6%	0.2%	0.2%
Fraudulent Return n=1,488	0%	0.1%	99.9%	0%
Wardrobing n=1,131	0%	0.1%	0%	99.9%

↑ Actual class (test set, n=14,888 total)

Diagonal-dominant, as expected from the headline F1 — read alongside the caveat below rather than as evidence the leakage concern is resolved: a synthetic near-deterministic label produces exactly this kind of clean diagonal.

Top feature importances (Random Forest)

days_to_return	0.152
wishlist_to_cart_time_hrs	0.126
return_rate_pct	0.117
return_to_order_ratio	0.115
tracking_number_valid	0.072
total_returns_lifetime	0.068
customer_support_contacts	0.044
refund_amount_requested_usd	0.043

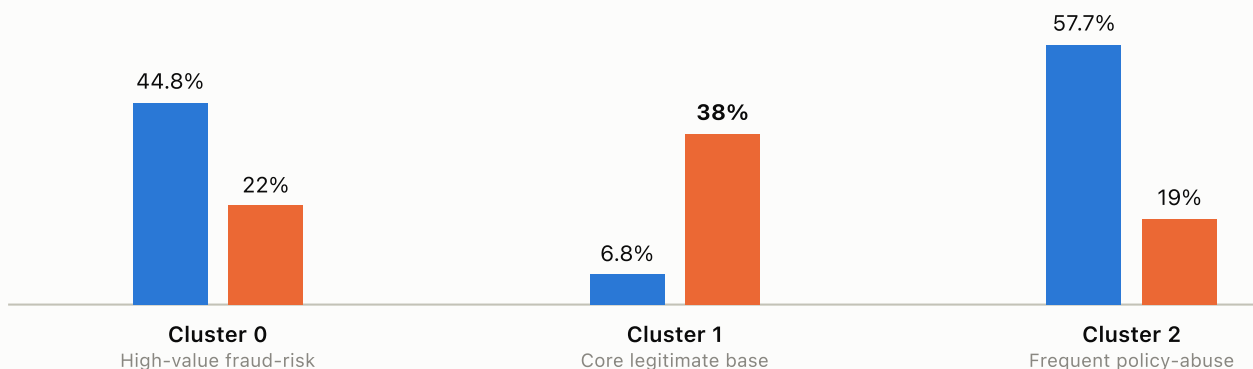
Model-evaluation caveat (see also `docs/fraud-classification-results.md`): weighted F1 of 0.998–0.999 is unrealistically high for a live fraud operation. Feature importance is spread across

~15 features with no single dominant leak — this isn't a one-column shortcut — but this is a **Kaggle synthetic dataset**, and synthetic label generators typically compose the target from a fairly clean rule, producing near-separable classes real transactional data never gives you. We present this as **proof of the pipeline and methodology**: the architecture, feature engineering, and evaluation approach transfer to production; this specific accuracy number does not. A realistic production target is **80–90% weighted F1**, and we recommend budgeting for that expectation, not this one, in any business case.

5.2 Approach 2 — K-Means clustering (quality & customer segmentation)

An independent, unsupervised lens: features deliberately **exclude the fraud-flag columns** used by the classifier, so clustering surfaces behavior/quality patterns a supervised fraud label can't. k=3 selected by silhouette score over k=2..6.

Cluster	n	Avg return rate	Avg order value	% quality-issue returns	Dominant mix
0 — High-value fraud-risk	6,646	44.8%	\$484.01	22%	Fraudulent Return-heavy
1 — Core legitimate base	43,829	6.8%	\$157.25	38%	96% Legitimate
2 — Frequent policy-abuse	9,525	57.7%	\$129.18	19%	Policy Abuser-heavy



■ Avg return rate ■ % of returns that are quality-issue driven

Cluster 1 has by far the lowest return rate but the *highest* quality-issue share — the two metrics move in opposite directions, which is exactly why a single blended "return rate" KPI hides this segment's problem.

Full weaknesses discussion (cluster stability, k-choice sensitivity, causal limits) in `docs/quality-clustering-results.md`.

6 · Business Insights & Strategic Recommendations

1. **Require photo evidence above a value threshold — ship this before the model.** The "high-value item + no photo evidence" pattern alone touches **\$1.88M** of non-legitimate refund exposure (4,673 orders) and needs no ML: a policy rule, live immediately, independent of the classifier's rollout timeline.
2. **Deploy the fraud classifier as a triage score, not an auto-block.** Route the highest `fraud_signal_score` decile plus Cluster 0 members to manual fraud review; auto-approve the lowest-risk tail. Re-validate precision/recall on live traffic before trusting this dataset's 0.999 F1 — budget for 80–90% in the initial production SLA. Prioritize Fraudulent Return claims specifically: at \$434.73 average vs. \$141.14 for legitimate returns, each one caught is worth ~3x a typical legitimate claim in review effort saved.
3. **Give Cluster 1 (core legitimate, 73% of customers) to QA, not fraud.** Its 38% quality-issue-return rate is the highest of the three segments despite near-zero fraud — cross-reference dominant `product_category` within this cluster against `return_reason` to identify which vendors/categories need a quality escalation.
4. **Tighten policy, don't investigate, for Cluster 2.** High order/return volume, low order value, Policy-Abuser-dominated — a shorter return window or restocking fee targets this segment more precisely than a fraud investigation would.
5. **Re-threshold before deployment.** A false "Legitimate → Fraudulent Return" flag costs more in customer trust than a slow manual review costs in ops time — set the classification threshold asymmetrically, not at a default 0.5, once real cost estimates are available.
6. **Re-validate quarterly.** Fraud patterns adapt to whatever the model learns; without a temporal holdout in this dataset, we cannot measure drift here — production deployment needs a rolling time-based validation window from day one.

Risks & future roadmap

- **Risk — synthetic-data overfit to production:** mitigated by treating this model as a pipeline proof-of-concept and re-training on real labeled fraud outcomes before go-live.
- **Risk — no ground-truth fraud labels in production:** real retailers rarely have a clean `abuse_type` column; roadmap includes a human-in-the-loop labeling process from the manual review queue this system creates.
- **Future:** real-time scoring at return-request time (not batch), a feedback loop from review-team overrides back into re-training, and extending clustering features with unstructured support-ticket text once the Flume/API ingestion (Section 3) is live.

Appendix

A. AI usage disclosure

Claude (Anthropic) was used throughout this project as an AI pair-programmer and analyst: drafting the PySpark data-engineering and MLlib modeling notebooks, generating this report and the accompanying executive deck, and proposing the feature-engineering rationale in Section 4.3. All code was executed end-to-end against the real 60,000-row dataset (not fabricated output) and every metric quoted in this report was read directly from that execution — see `docs/fraud-classification-results.csv`, `docs/rf-feature-importance.csv`, and `docs/quality-clustering-results.csv` for the underlying numbers. Team members reviewed, validated, and take ownership of the business framing, architecture decisions, and final recommendations.

B. Team contributions

Name	Roll no.	Workstream	Artifacts owned
TBD	TBD	Data acquisition & Bronze/Silver pipeline	<code>notebooks/01_data_engineering.ipynb</code>
TBD	TBD	Feature engineering & Gold layer	<code>notebooks/01_data_engineering.ipynb</code> §4
TBD	TBD	Classification modeling & evaluation	<code>notebooks/02_ml_modeling.ipynb</code> §2
TBD	TBD	Clustering & segmentation	<code>notebooks/02_ml_modeling.ipynb</code> §3
TBD	TBD	Architecture & enterprise design	Report §3
TBD	TBD	Narrative, report & deck	This report; presentation deck

Fill in names/roll numbers before submission — the same split must match the team table in `README.md`.

C. References

- Dataset: `ecommerce_return_abuse_dataset.csv`, Kaggle.
- Apache Spark MLlib documentation — classification, clustering, and feature transformers.
- Project brief: `docs/ProjectGuideLines_Big_Data_Analytics_Project_Guidelines.pdf`.